EECS 4314 E - Advanced Software Engineering
Project Design - Group 10

Authors/Contributors:
- Manjot Kaur
- Gurnoor Kahlon
- Nathan Kwok
- Andriy Lytvyn
- Larissa Singh
- Rishab Yadav

Last updated: June 14, 2026

Status: Final

Purpose

There is a growing demand for guidance in cooking on the internet. Savr is designed to make recipe sharing and food discovery more accessible and engaging. While many recipes are currently shared through personal blogs, websites, and social media accounts, it can be difficult for creators to reach new audiences and for users to discover and keep track of content from multiple sources. Savr addresses this problem by providing a centralized social media platform where users can share recipes, explore new ideas, and interact with other members of the cooking community. Features such as following creators, saving recipes, commenting, and rating content allow users to stay connected with their favorite bloggers while also discovering new ones. This design document outlines the architecture, features, and implementation plan for Savr and serves as a guide for the development of the system.

Background Reading

During the planning and design of Savr, our team looked at a variety of existing platforms and technologies to better understand how users discover content, interact with creators, and engage with online communities. We examined YouTube's subscriber notification system as a reference for our follow feature, since it provides an effective way for users to stay updated when creators publish new content. This helped us think about how recipe creators and followers could remain connected through the platform.

We also explored Reddit's content organization and discussion structure for ideas on how recipes, comments, and user interactions could be presented. Reddit's approach to user-generated content influenced several aspects of our design, particularly how users browse content, participate in discussions, and discover new creators and communities.

From a technical perspective, our design was influenced by the Service-Based Architecture concepts discussed in class, particularly the Software Architecture Styles (Distributed

Systems) lecture material. We also considered other architectural styles, including a traditional monolithic architecture and a full microservices architecture. A monolithic design would be simpler to implement, but it would make it harder to separate responsibilities and distribute work across the team. A full microservices approach would provide stronger independence between services, but it would add unnecessary complexity for the scope and timeline of this course project. For these reasons, we selected a service-based architecture because it provides a practical balance between modularity, maintainability, and implementation effort. We also consulted the official documentation for React, Spring Boot, MySQL, GitHub, and Google Custom Search API to better understand the technologies that will be used during development. These resources helped shape both the technical architecture and the user experience of Savr, and provided a foundation for many of the design decisions.

Context

Savr is designed to provide a centralized platform for recipe sharing, food discovery, and community engagement. While recipes are currently spread across the internet on personal blogs, websites, and social media platforms, users often need to search across multiple sources to discover new content and stay connected with their favourite creators. Savr aims to bring these experiences together by creating a dedicated platform where users can share recipes, discover cooking ideas, and interact with other members of the community. The concept for Savr was influenced by our research into existing content-sharing platforms such as YouTube and Reddit. These platforms demonstrate how content discovery, creator-focused communities, and user interaction can encourage long-term engagement. These ideas helped shape many of the features planned for Savr, including following creators, interacting through comments and ratings, and providing users with an easy way to discover new content.

The decision to use a service-based approach was influenced by the need for maintainability, future expansion, and effective distribution of development work among team members. The API layer provides an additional level of separation between users and backend services, helping improve security and manage communication throughout the system.

The website is intended to cultivate a collection of cooking-focused communities where users can explore recipes, share ideas, and communicate with one another. Recipe discovery is a major focus of the platform, with content being easily accessible through the home page while additional functionality is organized into dedicated sections of the application. This approach encourages exploration while keeping the user experience intuitive and organized.

From a broader business perspective, the platform has been designed with future growth in mind. A potential long-term business model could be advertisement-based, where companies pay to promote products or services to users while they browse content. Popular creators could potentially receive compensation based on engagement and viewership. Users may also be able to include links to donation platforms, e-commerce websites, or other social media accounts within their profiles. While these features will not be implemented as part of the

course project, they were considered during the design process to ensure the platform can be extended in the future without significant architectural changes.

The overall goal of Savr is to create an engaging and user-friendly environment that makes recipe sharing and food discovery more accessible while supporting the growth of creator-driven cooking communities.

Detailed Design

Savr will follow a service-based architecture style with a single monolithic user interface. The system will consist of a React frontend connected to multiple backend services developed using Spring Boot, with MySQL used for persistent data storage. The frontend layer will provide a single user interface where users can interact with the application by creating accounts, logging in, uploading recipes, viewing recipes, commenting, rating posts, and following other users. The interface will also allow users to sort recipes by highest rating, most viewed, newest uploads within the last week, or all-time popularity. The backend service layer will handle the main application logic and communication with the database.

System Components:

- The React frontend provides the main user interface for the platform. Users will use it to register, log in, browse recipes, search content, upload recipes, save recipes, comment, rate posts, follow creators, and view recipe statistics.
- The API Gateway acts as the central entry point for requests from the frontend. It routes requests to the appropriate backend services and helps separate the user interface from the backend business logic. The API Gateway also simplifies communication between the frontend and backend by providing a single access point for system requests.
- The Authentication Service handles user registration, login, logout, session management, and authentication validation.
- The Recipe Service handles recipe creation, editing, deletion, viewing, ingredient management, and recipe image information.
- The Comment and Rating Service handles adding, editing, and deleting comments, as well as submitting and calculating recipe ratings.
- The Follow Service handles following and unfollowing users, viewing follower/following lists, and supporting creator-based content discovery.
- The Saved Recipe Service handles saving recipes, removing saved recipes, and displaying a user's saved recipe collection.
- The Sorting and Statistics Service handles sorting recipes by newest, most viewed, highest rated, and trending. It also tracks basic recipe statistics such as views and ratings.
- The Image Service retrieves ingredient-related images through the Google Custom Search API.

- The MySQL operational database stores the main application data, including users, recipes, ingredients, comments, ratings, followers, and saved recipes.
- The analytics/data lakehouse layer is included as a planned extension for storing user activity data such as recipe views, clicks, searches, ratings, and follow events. For the course implementation, this may be simplified to basic activity logging and statistics tracking.

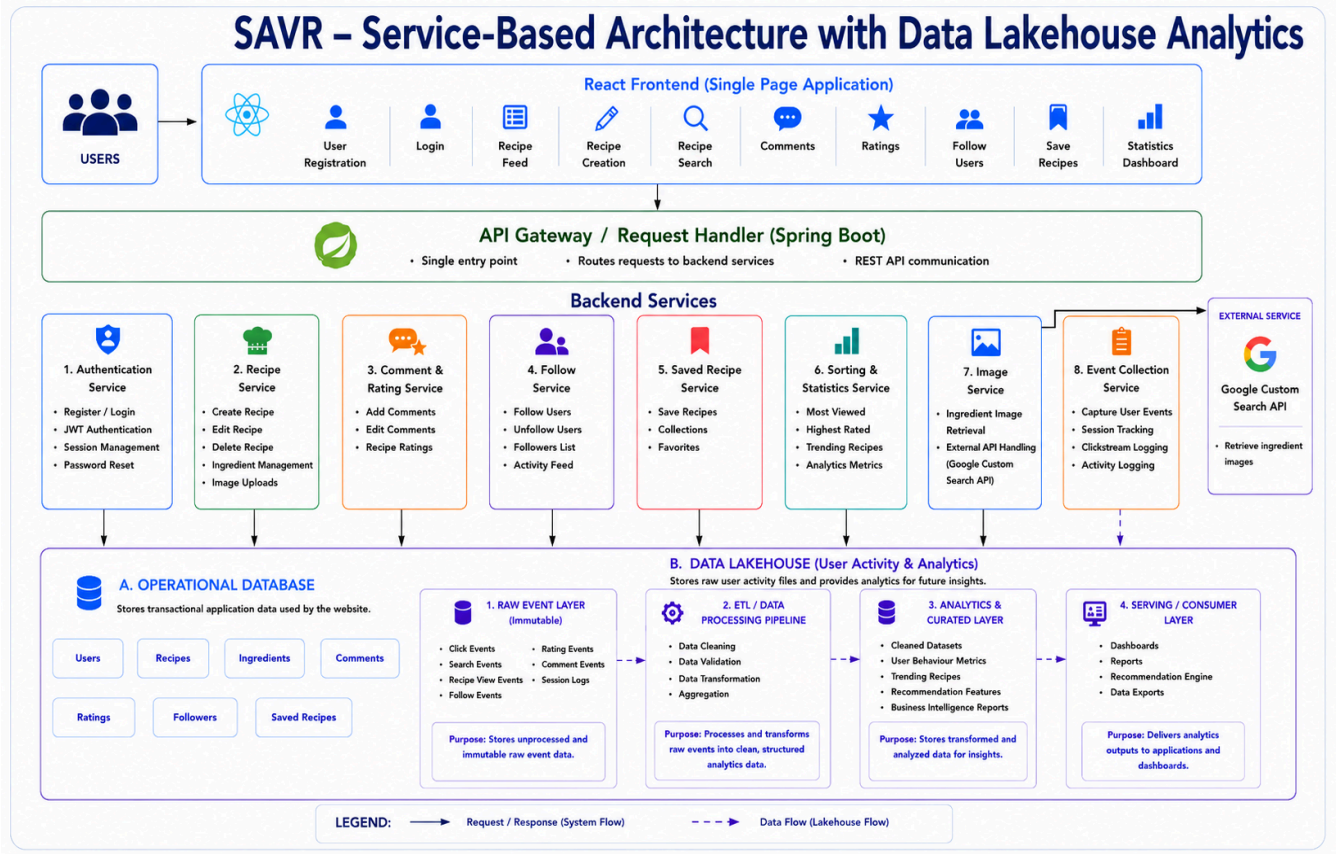

Figure 1: Service-based architecture of Savr

How Users Interact with the System:

Users interact with Savr through the React web application. A typical user begins by creating an account or logging in. After logging in, the user can browse the recipe feed, search for recipes, view recipe details, save recipes, leave comments, submit ratings, and follow other users. Creators can upload recipes by entering a recipe title, ingredients, instructions, images, and optional tags or categories. Once submitted, the recipe becomes available for other users to view and interact with. Users can also sort and filter recipes based on criteria such as newest uploads, highest rated, most viewed, and trending content. This supports recipe discovery and makes it easier for users to find content that matches their interests.

How Software Components Interact:

When a user performs an action, the React frontend sends a request to the API Gateway. The API Gateway forwards the request to the backend service responsible for that action. For example, login requests are handled by the Authentication Service, recipe uploads are handled by the Recipe Service, comments and ratings are handled by the Comment and Rating Service, and saved recipe requests are handled by the Saved Recipe Service. Each backend service performs its own business logic and communicates with the MySQL database when data needs to be stored or retrieved. The Image Service also communicates with the external Google Custom Search API when ingredient-related images are needed. This structure keeps each service focused on a specific responsibility while still allowing the system to function as one complete application.

How Data Flows Through the System:

When a user uploads a recipe, the recipe information is entered through the frontend and sent to the API Gateway. The API Gateway routes the request to the Recipe Service. The Recipe Service validates the recipe information and stores it in the MySQL database. Once saved, the recipe can be returned to the frontend and displayed in the recipe feed or recipe detail page. When a user comments on or rates a recipe, the request is sent from the frontend to the API Gateway and then to the Comment and Rating Service. The service stores the comment or rating in the database and updates the recipe's displayed information. When a user follows another user or saves a recipe, the request follows the same general path: frontend to API Gateway, then to the appropriate backend service, then to the database. The updated result is returned to the frontend so the user can see the change.

User activity such as recipe views, searches, ratings, and follows may also be collected for analytics purposes. This activity data can be stored separately from the main operational database so it can later be used for reports, dashboards, and trend analysis. Overall, the data flow separates user-facing application data from analytical activity data, while keeping the system practical and manageable for implementation.

Implementation Plan

Our team will build Savr in stages so that each major part of the system can be developed, tested, and integrated gradually. The project will begin with the core setup, including the frontend structure, backend project setup, and MySQL database schema. After that, each service will be implemented individually and connected to the frontend and database as development progresses.

Technologies We Intend to Use:

The frontend will be developed using React, JavaScript, HTML, and CSS. The backend will be developed using Java Spring Boot, with MySQL used for persistent storage. Development will be done using Visual Studio Code and Eclipse. GitHub and GitHub Desktop will be used

for collaboration and version control. A simplified data lakehouse or analytics layer will also be used for storing user activity data, such as recipe views, ratings, searches, and other interaction events. If this becomes too large for the project timeline, it will be simplified into basic activity logging within the database.

Source Code Management:

Source code will be managed using GitHub. The team will use GitHub Desktop or Git commands to commit and push changes. Each team member will work on assigned features using separate branches, and completed work will be merged into the main project after review. The repository will be organized into separate folders for the frontend, backend, database scripts, and documentation. This will help keep the project organized and make it easier for team members to work on different parts of the system without interfering with each other's work.

Component Integration:

Each service will be developed individually before being connected to the rest of the system. For example, the authentication service, recipe service, comment and rating service, follow service, and saved recipe service will each be implemented and tested separately. Once a service is working on its own, it will be connected to the MySQL database and then integrated with the React frontend through API calls. Integration will happen gradually so that errors can be found and fixed early instead of waiting until the end of the project.

Testing Plan:

The system will be tested using a combination of unit testing, integration testing, system testing, and manual frontend testing. Unit testing will be used to test individual backend functions and service logic. Integration testing will check that the frontend, backend services, and database communicate correctly. System testing will be used to test complete user workflows, such as registration, login, uploading a recipe, commenting, rating, saving a recipe, and following another user. Frontend components will also be manually tested to check layout, navigation, form validation, and overall usability. Testing will be repeated after major integrations to make sure new changes do not break existing features.

Deployment Plan:

During development and testing, Savr will be deployed locally using localhost. The React frontend, Spring Boot backend, and MySQL database will run locally so we can test the complete system before submission. If time allows, we may also explore hosting the application on a dedicated server or cloud platform. However, the main deployment plan for the course project is a working local deployment with clear setup instructions so the TA can run and review the system.

Project Planning

Team Organization:

Development responsibilities will be divided among team members based on the major services and components of the system. During the first phase of development, all team members will contribute to setting up the project structure, frontend foundation, and database design. After the initial setup, responsibilities will be divided across individual services to allow parallel development and reduce overlap. Each team member will be responsible for implementing, testing, and documenting their assigned functionality. Team members will also be responsible for maintaining clear code comments and updating project documentation as development progresses.

Who Does What?

The current plan is for each team member to take responsibility for one major service of the system. While responsibilities may change as development progresses, the current distribution of work is as follows:

- **Authentication Service** – User registration, login, and session management.
- **Recipe Service** – Recipe creation, editing, deletion, and viewing.
- **Comment and Rating Service** – Comments, ratings, and user interactions.
- **Follow Service** – Following and unfollowing users.
- **Saved Recipe Service** – Saving and managing recipe collections.
- **Statistics and Sorting Service** – Recipe statistics, trending content, and sorting functionality.
- **Image Service** – Integration with the Google Custom Search API.
- **Frontend and Database Development** – Shared responsibility across all team members during the early stages of development.

How Work Is Coordinated:

Team communication will primarily take place through a Discord group chat. The group chat will be used to assign work, discuss implementation decisions, report progress, and coordinate deadlines. Regular Discord voice calls will be scheduled for team meetings, progress reviews, and issue resolution. GitHub will be used to track development progress, manage code contributions, and review completed work before integration into the main branch. At least one team meeting will be held each week, with additional meetings scheduled as necessary before major deadlines and project check-ins.

Schedule:

The project schedule is organized around the course check-ins and final deliverables.

Weeks 1–2 (June 14 – June 26)

- Repository setup
- Frontend setup
- Database design and schema creation
- Authentication Service implementation
- Initial architecture implementation
- Preparation for Project Check-in #1

Milestone: Core project structure established and Authentication Service functional.

Project Check-in 1 (Week of June 26)

Deliverables:

- GitHub repository
- Progress report
- Individual contribution summary
- Working session with TA

Weeks 3–4 (June 27 – July 10)

- Recipe Service implementation
- Follow Service implementation
- Comment and Rating Service implementation
- Frontend integration for completed services
- Bug fixing and refinement

Milestone: Core social and recipe-sharing functionality operational.

Project Check-in 2 (Week of July 10)

Deliverables:

- Updated GitHub repository
- Progress report
- Demonstration of implemented functionality
- Working session with TA

Weeks 5–6 (July 11 – July 24)

- Saved Recipe Service implementation
- Statistics and Sorting Service implementation

- Image Service implementation
- System integration
- Additional frontend development
- Bug fixing and testing

Milestone: All major features implemented and integrated.

Project Check-in 3 (Week of July 24)

Deliverables:

- Fully integrated system prototype
- Updated progress report
- Working session with TA

Week of July 29

- Final testing
- Presentation preparation
- Demonstration rehearsal
- Project Presentation and Q&A

Milestone: Presentation-ready version of Savr completed.

Week of August 1

- Final bug fixes
- Final documentation updates
- Final report submission

Milestone: Project completion and final report submission.

Development Milestones:

Milestone 1 – Project Foundation

- Repository created
- Frontend initialized
- Database designed
- Authentication Service implemented

Milestone 2 – Core Functionality

- Recipe management completed
- Following functionality completed
- Commenting and rating completed

Milestone 3 – Feature Completion

- Saved recipes implemented
- Statistics and sorting implemented
- Image service integrated

Milestone 4 – System Integration

- All services connected
- Frontend and backend fully integrated
- Testing completed

Milestone 5 – Final Delivery

- Project presentation completed
- Final report submitted
- All project requirements satisfied

Appendix:

A. Preliminary Architecture Diagram

Figure A1 shows an early architecture sketch created during the planning phase of the project. The diagram was used to visualize the relationships between the frontend, API gateway, backend services, operational database, data lakehouse components, and external APIs before the final architecture was developed.

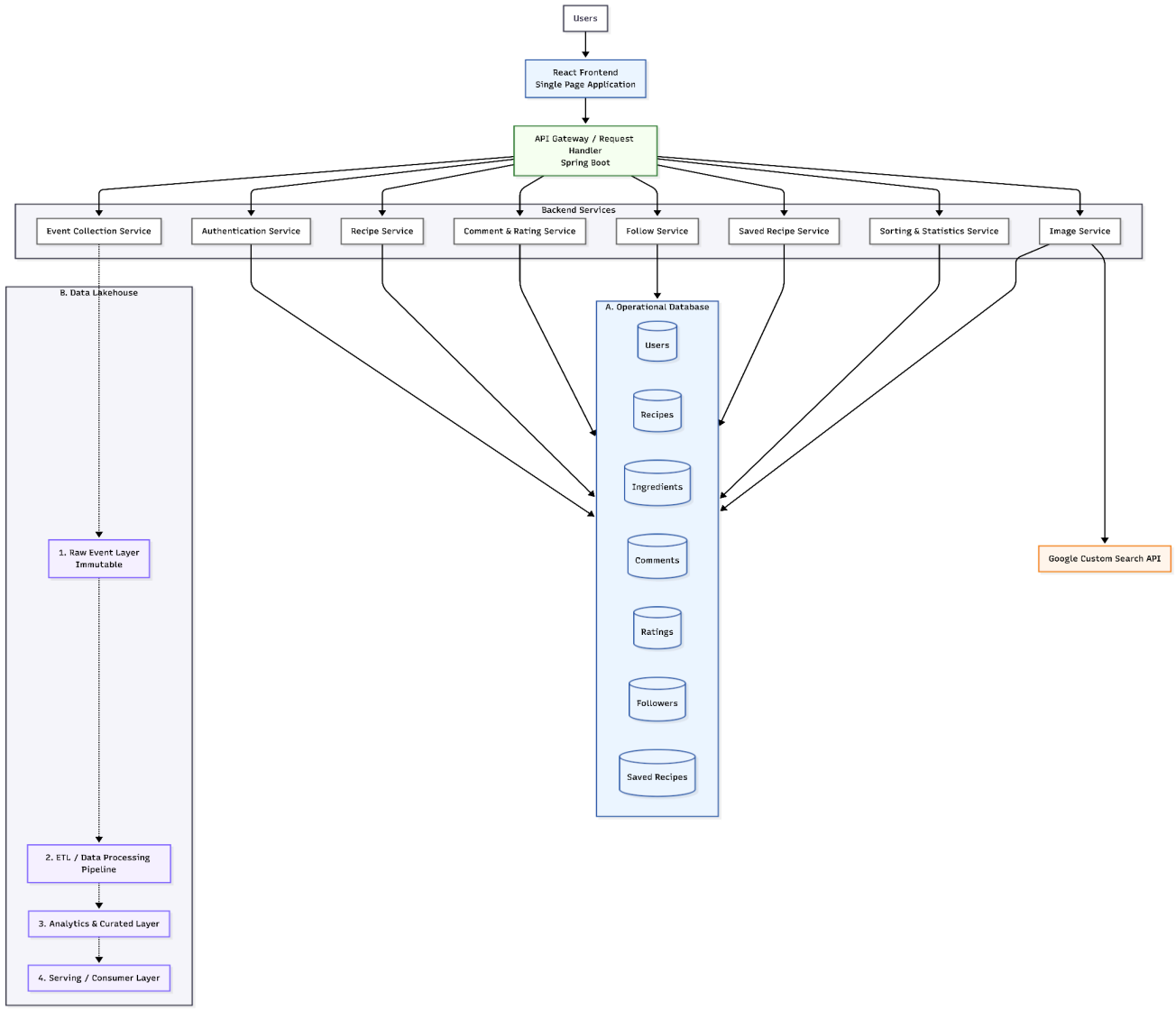

Figure A1: Preliminary architecture diagram used during the design phase.